

ES6+ JavaScript Guide

[ES6+ JavaScript — A Detailed Guide](#)

- [1. `let` and `const`](#)
 - [2. Arrow Functions](#)
 - [3. Template Literals](#)
 - [4. Destructuring](#)
 - [5. Default Parameters](#)
 - [6. Spread and Rest Operators \(`...` \)](#)
 - [7. Enhanced Object Literals](#)
 - [8. Classes](#)
 - [9. Modules \(`import` / `export` \)](#)
 - [10. Promises](#)
 - [11. `async` / `await`](#)
 - [12. Iterators and Generators](#)
 - [13. `Map` and `Set`](#)
 - [14. Symbols](#)
 - [15. Optional Chaining \(`?.` \) — ES2020](#)
 - [16. Nullish Coalescing \(`??` \) — ES2020](#)
 - [17. Array Methods](#)
 - [18. Object Methods](#)
 - [19. String Methods](#)
 - [20. `for...of` Loop](#)
 - [21. Proxy and Reflect](#)
 - [22. Newer Additions \(ES2021-ES2024\)](#)
- [Quick Reference Table](#)
- [Tips for practicing](#)

ES6+ JavaScript — A Detailed Guide

ES6 (ES2015) and later versions (ES2016 through ES2024) introduced major upgrades to JavaScript. This guide walks through the most important features with explanations and examples.

1. `let` and `const`

Before ES6, `var` was the only way to declare variables — function-scoped, hoisted, and re-declarable, which caused bugs.

```

// var: function-scoped, can be redeclared, hoisted with 'undefined'
var x = 1;
var x = 2; // no error

// let: block-scoped, cannot be redeclared in same scope
let y = 1;
y = 2; // OK, can be reassigned
// let y = 3; // Error: already declared

// const: block-scoped, cannot be reassigned
const z = 1;
// z = 2; // Error: assignment to constant

// Block scoping example
if (true) {
  let a = 10;
  var b = 20;
}
// console.log(a); // Error: a is not defined
console.log(b); // 20 - var leaks out of the block

```

Note: `const` prevents *reassignment*, not mutation:

```

const arr = [1, 2, 3];
arr.push(4); // fine - mutating the array
// arr = [5, 6]; // Error - reassigning the variable

```

2. Arrow Functions

Shorter syntax, and they don't have their own `this` (they inherit it from the enclosing scope).

```

// Traditional function
function add(a, b) {
  return a + b;
}

// Arrow function
const addArrow = (a, b) => a + b;

// Single parameter - parens optional
const square = x => x * x;

// No parameters
const greet = () => console.log('Hello!');

// Multi-line body needs braces + explicit return
const multiply = (a, b) => {
  const result = a * b;
  return result;
};

// 'this' binding difference
class Timer {
  constructor() {
    this.seconds = 0;
  }
  start() {
    // Arrow function inherits 'this' from start()
    setInterval(() => {
      this.seconds++;
      console.log(this.seconds);
    }, 1000);
  }
}

```

3. Template Literals

String interpolation and multi-line strings using backticks.

```
const name = 'Alice';
const age = 25;

// Interpolation
console.log(`My name is ${name} and I am ${age} years old.`);

// Expressions inside ${}
console.log(`Next year I'll be ${age + 1}.`);

// Multi-line strings
const message = `Line one
Line two
Line three`;

// Tagged templates (advanced)
function highlight(strings, ...values) {
  return strings.reduce((acc, str, i) =>
    `${acc}${str}${values[i] ? `**${values[i]}**` : ''}`, '');
}
console.log(highlight`Hello ${name}, you are ${age}`);
```

4. Destructuring

Extract values from arrays or objects into variables.

```

// Array destructuring
const numbers = [1, 2, 3];
const [first, second, third] = numbers;
console.log(first, second, third); // 1 2 3

// Skipping elements
const [, , onlyThird] = numbers;

// Swapping variables
let x = 1, y = 2;
[x, y] = [y, x];

// Default values
const [p = 10, q = 20] = [5];
console.log(p, q); // 5 20

// Object destructuring
const person = { name: 'Bob', age: 30, city: 'NYC' };
const { name, age } = person;

// Renaming
const { name: fullName } = person;

// Default values in objects
const { country = 'Unknown' } = person;

// Nested destructuring
const user = { id: 1, address: { city: 'LA', zip: '90001' } };
const { address: { city } } = user;

// Function parameter destructuring
function printUser({ name, age }) {
  console.log(`${name} is ${age}`);
}
printUser(person);

```

5. Default Parameters

```

function greetUser(name = 'Guest', greeting = 'Hello') {
  console.log(`${greeting}, ${name}!`);
}
greetUser(); // Hello, Guest!
greetUser('Sam'); // Hello, Sam!

// Defaults can reference earlier parameters
function createBox(width, height = width) {
  return { width, height };
}

```

6. Spread and Rest Operators (...)

Spread expands an iterable; **rest** collects arguments into an array.

```

// Spread in arrays
const arr1 = [1, 2, 3];
const arr2 = [...arr1, 4, 5]; // [1, 2, 3, 4, 5]

// Spread in function calls
function sum(a, b, c) { return a + b + c; }
console.log(sum(...[1, 2, 3])); // 6

// Spread in objects (ES2018)
const obj1 = { a: 1, b: 2 };
const obj2 = { ...obj1, c: 3 }; // { a: 1, b: 2, c: 3 }

// Cloning
const clonedArr = [...arr1];
const clonedObj = { ...obj1 };

// Rest parameters – gathers remaining args into an array
function multiplyAll(multiplier, ...numbers) {
  return numbers.map(n => n * multiplier);
}
console.log(multiplyAll(2, 1, 2, 3)); // [2, 4, 6]

// Rest in destructuring
const [head, ...tail] = [1, 2, 3, 4];
console.log(head, tail); // 1 [2, 3, 4]

const { a, ...rest } = { a: 1, b: 2, c: 3 };
console.log(rest); // { b: 2, c: 3 }

```

7. Enhanced Object Literals

```

const name = 'Alice';
const age = 25;

// Shorthand property names
const person = { name, age }; // instead of { name: name, age: age }

// Shorthand methods
const calculator = {
  add(a, b) { return a + b; }, // instead of add: function(a, b) {...}
  subtract(a, b) { return a - b; }
};

// Computed property names
const key = 'dynamicKey';
const obj = {
  [key]: 'value',
  [`_${key}_2`]: 'value2'
};

```

8. Classes

Syntactic sugar over prototype-based inheritance.

```

class Animal {
  // Class fields (ES2022)
  species = 'Unknown';
  #secret = 'private'; // private field

  constructor(name) {
    this.name = name;
  }

  speak() {
    console.log(`${this.name} makes a sound.`);
  }

  static create(name) { // static method
    return new Animal(name);
  }

  get info() { // getter
    return `${this.name} (${this.species})`;
  }

  set info(value) { // setter
    this.species = value;
  }

  revealSecret() {
    return this.#secret;
  }
}

class Dog extends Animal {
  constructor(name, breed) {
    super(name); // call parent constructor
    this.breed = breed;
  }

  speak() { // method overriding
    super.speak(); // call parent method
    console.log(`${this.name} barks.`);
  }
}

const rex = new Dog('Rex', 'Labrador');
rex.speak();
console.log(rex.info);

```

9. Modules (import / export)

```

// math.js - named exports
export const PI = 3.14159;
export function add(a, b) { return a + b; }
export function subtract(a, b) { return a - b; }

// default export
export default function multiply(a, b) { return a * b; }

// main.js - importing
import multiply, { PI, add, subtract } from './math.js';
import * as MathUtils from './math.js'; // namespace import

console.log(add(2, 3));
console.log(MathUtils.PI);

```

10. Promises

Handle asynchronous operations without deeply nested callbacks.

```
function fetchData(shouldSucceed) {
  return new Promise((resolve, reject) => {
    setTimeout(() => {
      if (shouldSucceed) {
        resolve('Data received');
      } else {
        reject(new Error('Failed to fetch'));
      }
    }, 1000);
  });
}

fetchData(true)
  .then(data => console.log(data))
  .catch(error => console.error(error))
  .finally(() => console.log('Done'));

// Chaining
fetchData(true)
  .then(data => data.toUpperCase())
  .then(upper => console.log(upper));

// Promise.all — wait for all to resolve
Promise.all([fetchData(true), fetchData(true)])
  .then(results => console.log(results));

// Promise.race — first to settle wins
Promise.race([fetchData(true), fetchData(false)])
  .then(console.log)
  .catch(console.error);

// Promise.allSettled (ES2020) — waits for all, never rejects
Promise.allSettled([fetchData(true), fetchData(false)])
  .then(results => console.log(results));
```

11. async / await

Syntactic sugar over Promises — write async code that reads like sync code.

```
async function getData() {
  try {
    const data = await fetchData(true);
    console.log(data);
    return data;
  } catch (error) {
    console.error('Error:', error.message);
  }
}

// Running multiple async tasks in parallel
async function getAllData() {
  const [result1, result2] = await Promise.all([
    fetchData(true),
    fetchData(true)
  ]);
  console.log(result1, result2);
}

// Top-level await (ES2022, in modules)
const data = await fetchData(true);
```

12. Iterators and Generators

```
// Generator function – produces an iterator
function* numberGenerator() {
  yield 1;
  yield 2;
  yield 3;
}

const gen = numberGenerator();
console.log(gen.next()); // { value: 1, done: false }
console.log(gen.next()); // { value: 2, done: false }
console.log([...numberGenerator()]); // [1, 2, 3]

// Infinite generator
function* idGenerator() {
  let id = 1;
  while (true) {
    yield id++;
  }
}

const ids = idGenerator();
console.log(ids.next().value); // 1
console.log(ids.next().value); // 2

// Custom iterable
const range = {
  from: 1,
  to: 5,
  [Symbol.iterator]() {
    let current = this.from;
    const last = this.to;
    return {
      next() {
        return current <= last
          ? { value: current++, done: false }
          : { value: undefined, done: true };
      }
    };
  }
};

console.log([...range]); // [1, 2, 3, 4, 5]
```

13. Map and Set


```

// Map – key-value pairs with any type of key
const map = new Map();
map.set('name', 'Alice');
map.set(1, 'one');
map.set(true, 'boolean key');

console.log(map.get('name')); // Alice
console.log(map.size); // 3

for (const [key, value] of map) {
  console.log(key, value);
}

// Set – collection of unique values
const set = new Set([1, 2, 2, 3, 3, 3]);
console.log(set); // Set(3) {1, 2, 3}
set.add(4);
set.has(2); // true

// Removing duplicates from an array
const uniqueArr = [...new Set([1, 1, 2, 2, 3])]; // [1, 2, 3]

// WeakMap / WeakSet – keys are weakly referenced (garbage-collectable)
const weakMap = new WeakMap();
let obj = { id: 1 };
weakMap.set(obj, 'metadata');

```

14. Symbols

Unique, immutable primitive values often used as object keys to avoid collisions.

```

const sym1 = Symbol('id');
const sym2 = Symbol('id');
console.log(sym1 === sym2); // false – always unique

const user = {
  name: 'Alice',
  [sym1]: '12345' // hidden-ish property
};
console.log(user[sym1]); // 12345

```

15. Optional Chaining (?.) — ES2020

Safely access nested properties without manual null checks.

```

const user = {
  name: 'Alice',
  address: { city: 'NYC' }
};

console.log(user?.address?.city); // NYC
console.log(user?.contact?.phone); // undefined (no error)

// Works with function calls too
user.greet?.(); // does nothing if greet doesn't exist

// Works with array access
const arr = null;
console.log(arr?.[0]); // undefined

```

16. Nullish Coalescing (??) — ES2020

Returns the right-hand value only when the left is `null` or `undefined` (unlike `||`, which also triggers on `0`, `''`, `false`).

```
const count = 0;
console.log(count || 10); // 10 - wrong! 0 is falsy
console.log(count ?? 10); // 0 - correct! 0 is not nullish

let username;
console.log(username ?? 'Anonymous'); // Anonymous

// Combined with optional chaining
const city = user?.address?.city ?? 'Unknown';

// Logical nullish assignment (ES2021)
let config = { theme: null };
config.theme ??= 'dark';
console.log(config.theme); // dark
```

17. Array Methods

```
const nums = [1, 2, 3, 4, 5];

nums.map(n => n * 2);           // [2, 4, 6, 8, 10]
nums.filter(n => n % 2 === 0);  // [2, 4]
nums.reduce((sum, n) => sum + n, 0); // 15
nums.find(n => n > 3);           // 4
nums.findIndex(n => n > 3);       // 3
nums.some(n => n > 4);           // true
nums.every(n => n > 0);          // true
nums.includes(3);               // true (ES2016)

// flat / flatMap (ES2019)
[1, [2, 3, [4, 5]]].flat();     // [1, 2, 3, [4, 5]]
[1, [2, 3, [4, 5]]].flat(2);    // [1, 2, 3, 4, 5]
[1, 2, 3].flatMap(n => [n, n * 2]); // [1, 2, 2, 4, 3, 6]

// Array.from - create array from iterable/array-like
Array.from({ length: 5 }, (_, i) => i * 2); // [0, 2, 4, 6, 8]
Array.from('hello'); // ['h', 'e', 'l', 'l', 'o']

// Array.of
Array.of(7); // [7] (vs Array(7) which makes empty length-7 array)

// at() (ES2022) - supports negative indices
nums.at(-1); // 5
```

18. Object Methods

```

const obj1 = { a: 1, b: 2 };
const obj2 = { b: 3, c: 4 };

Object.assign({}, obj1, obj2);    // { a: 1, b: 3, c: 4 }
Object.keys(obj1);                // ['a', 'b']
Object.values(obj1);              // [1, 2]
Object.entries(obj1);              // [['a', 1], ['b', 2]]

Object.fromEntries([['a', 1], ['b', 2]]); // { a: 1, b: 2 } (ES2019)

Object.freeze(obj1);              // makes immutable
Object.isFrozen(obj1);            // true

// hasOwn (ES2022) – safer replacement for hasOwnProperty
Object.hasOwn(obj1, 'a');          // true

```

19. String Methods

```

'hello'.includes('ell');          // true
'hello'.startsWith('he');        // true
'hello'.endsWith('lo');          // true
'hi'.repeat(3);                  // 'hihihi'
' hi '.trim();                   // 'hi'
' hi '.trimStart();              // 'hi ' (ES2019)
' hi '.trimEnd();                // ' ' hi' (ES2019)
'5'.padStart(3, '0');            // '005'
'5'.padEnd(3, '0');              // '500'

// replaceAll (ES2021)
'a-b-c'.replaceAll('-', '_');    // 'a_b_c'

// matchAll (ES2020)
[...'a1b2c3'.matchAll(/[a-z](\d)/g)].map(m => m[1]); // ['1', '2', '3']

```

20. for...of Loop

Iterates over any iterable (arrays, strings, Maps, Sets), unlike `for...in`, which iterates over keys.

```

for (const num of [1, 2, 3]) console.log(num);
for (const char of 'abc') console.log(char);

const map = new Map([['a', 1], ['b', 2]]);
for (const [key, value] of map) console.log(key, value);

// With entries()
for (const [index, value] of ['x', 'y'].entries()) {
  console.log(index, value); // 0 x, 1 y
}

```

21. Proxy and Reflect

```
const target = { name: 'Alice', age: 25 };

const handler = {
  get(obj, prop) {
    console.log(`Getting ${prop}`);
    return Reflect.get(obj, prop);
  },
  set(obj, prop, value) {
    console.log(`Setting ${prop} to ${value}`);
    return Reflect.set(obj, prop, value);
  }
};

const proxy = new Proxy(target, handler);
proxy.name; // logs: Getting name
proxy.age = 30; // logs: Setting age to 30
```

22. Newer Additions (ES2021-ES2024)

```
// Numeric separators (ES2021) – for readability
const billion = 1_000_000_000;

// Array.prototype.group / groupBy candidates aside, here are stable ones:

// structuredClone (ES2022, widely available) – deep clone
const original = { a: 1, nested: { b: 2 } };
const copy = structuredClone(original);

// Array find from the end (ES2023)
[1, 2, 3, 4].findLast(n => n % 2 === 0); // 4
[1, 2, 3, 4].findLastIndex(n => n % 2 === 0); // 3

// toSorted, toReversed, toSpliced, with() (ES2023) – non-mutating versions
const sorted = [3, 1, 2].toSorted(); // [1, 2, 3], original unchanged
const reversed = [1, 2, 3].toReversed(); // [3, 2, 1]
const replaced = [1, 2, 3].with(1, 99); // [1, 99, 3]

// Array grouping (ES2024)
const grouped = Object.groupBy([1, 2, 3, 4, 5], n => n % 2 === 0 ? 'even' : 'odd');
// { odd: [1, 3, 5], even: [2, 4] }

// Promise.withResolvers (ES2024)
const { promise, resolve, reject } = Promise.withResolvers();
```

Quick Reference Table

Feature	Version	Purpose
let / const	ES6	Block-scoped variables
Arrow functions	ES6	Concise functions, lexical this
Template literals	ES6	String interpolation
Destructuring	ES6	Unpack arrays/objects
Spread/Rest	ES6	Expand/collect values
Classes	ES6	OOP syntax
Modules	ES6	import / export
Promises	ES6	Async handling
async / await	ES2017	Sync-style async code

<code>Object.values/entries</code>	ES2017	Object iteration
<code>Array.includes</code>	ES2016	Membership check
Rest/spread for objects	ES2018	Object spread
<code>Promise.allSettled</code>	ES2020	Wait for all promises
Optional chaining <code>?.</code>	ES2020	Safe property access
Nullish coalescing <code>??</code>	ES2020	Default for null/undefined
<code>String.replaceAll</code>	ES2021	Replace all occurrences
Logical assignment (<code>??=</code>)	ES2021	Conditional assignment
Class private fields <code>#</code>	ES2022	Encapsulation
<code>Array.at()</code>	ES2022	Negative indexing
<code>structuredClone</code>	ES2022	Deep cloning
<code>Array.findLast</code>	ES2023	Search from end
<code>toSorted</code> / <code>with</code> etc.	ES2023	Non-mutating array methods
<code>Object.groupBy</code>	ES2024	Group array items

Tips for practicing

1. Try rewriting old `var / function` code using `let / const` and arrow functions.
2. Practice destructuring function parameters — it's extremely common in React and Node.js code.
3. Build a small async project (e.g., fetch data from an API) using `async / await`.
4. Experiment with `Map / Set` instead of plain objects/arrays when order and uniqueness matter.